المملكة المغربية
جامعة عبد المالك السعدي
المدرسة الوطنية للعلوم التطبيقية بالحسيمة

Royaume du Maroc
Université Abdelmalek Essaâdi
Ecole Nationale des Sciences
Appliquées d'Al Hoceima

ⵜⴰⴳⵍⴷⵉⵜ ⵏ ⵍⵎⵖⵔⵉⴱ
ⵜⴰⵙⴷⴰⵡⵉⵜ ⵄⴰⴱⴷⵍⵎⴰⵍⵉⴽ ⵙⵙⴰⴷⵉ
ⵜⵉⵏⵎⵍ ⵜⴰⵏⴰⵎⵓⵔⵜ ⵏ ⵜⵎⴰⵙⵉⵍⵜ
ⵜⵓⵏⵉⵙⵉⵏ ⵏ ⵍⵄⵓⵙⵎⴰⵏⵉ

SPRING FRAMEWORK ET MICRO SERVICES

Pr. Achraf Boumhidi

a.boumhidi@uae.ac.ma

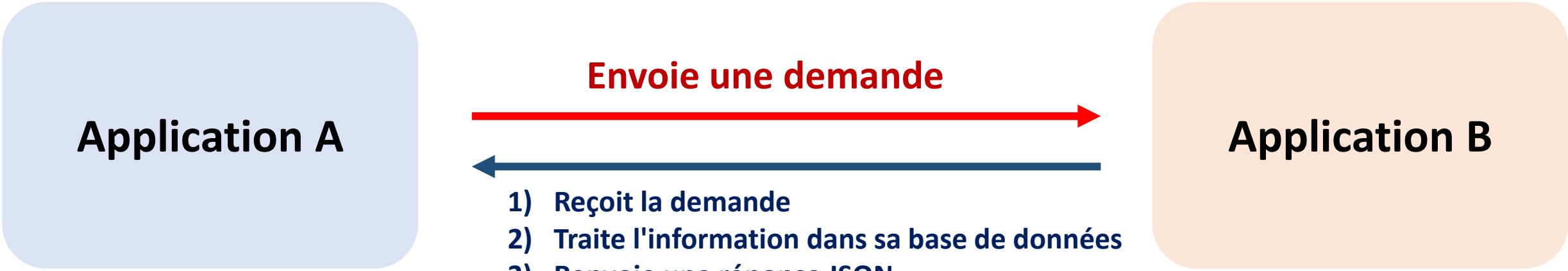
Développement des API REST

API : Application Programming Interface

REST : Representational State Transfer

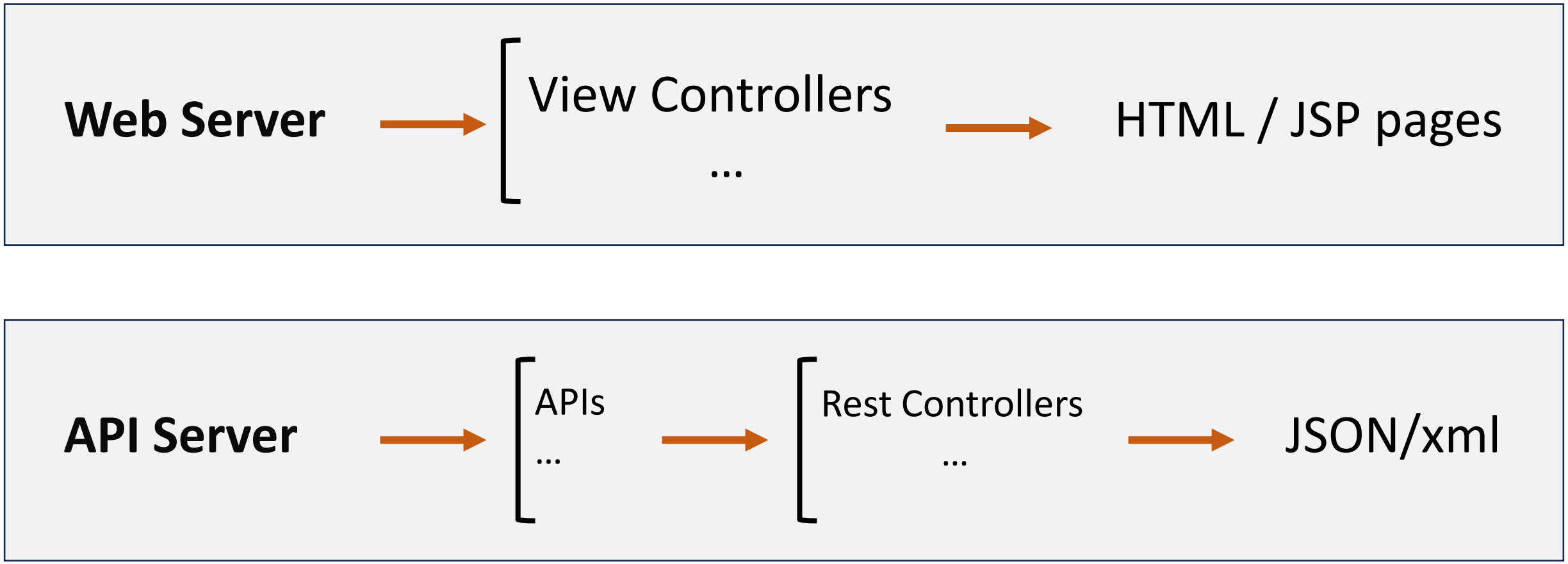
Une application Spring Boot peut jouer deux rôles : elle est un serveur Web (servant des pages) et un **serveur API** (servant des données).

Une **API** est une **interface** qui permet à deux applications de communiquer et d'échanger des données de manière automatique.



- application React
- application mobile iOS

- **serveur API**



- Par définition, une classe annotée avec @RestController est une Web API.

Exemple d’un Restfull API controller

```java
@RestController
@RequestMapping("/api/movies")
public class MovieApiController {

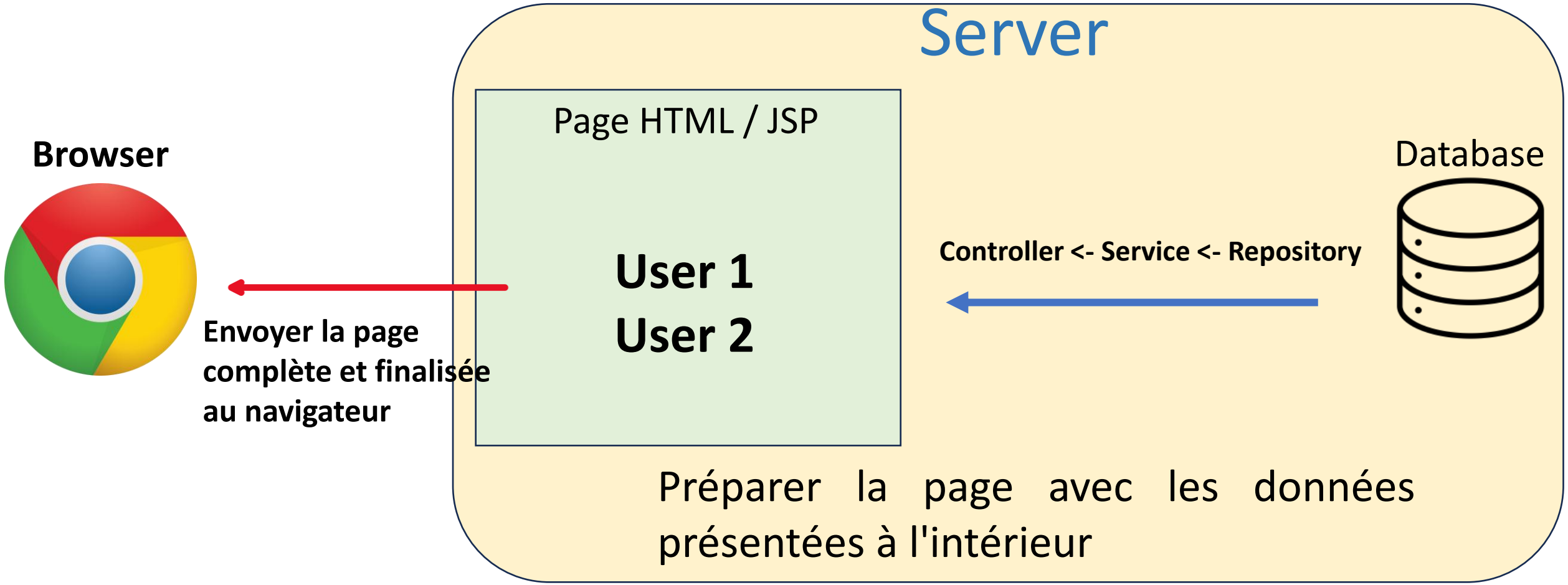
@Autowired
private MovieService service;


@GetMapping
public List<Movie> getMovies() {
return service.getAllMovies();
}
```

Why we use Rest Controllers and JSON responses ?

SSR (Server-Side Rendering)

- Le serveur prépare la page HTML **complète** avec toutes les données déjà intégrées avant de l'envoyer au navigateur.



SSR (Server-Side Rendering)

Avantages :

- ❑ Excellent pour le référencement (SEO) car Google reçoit du contenu déjà prêt.

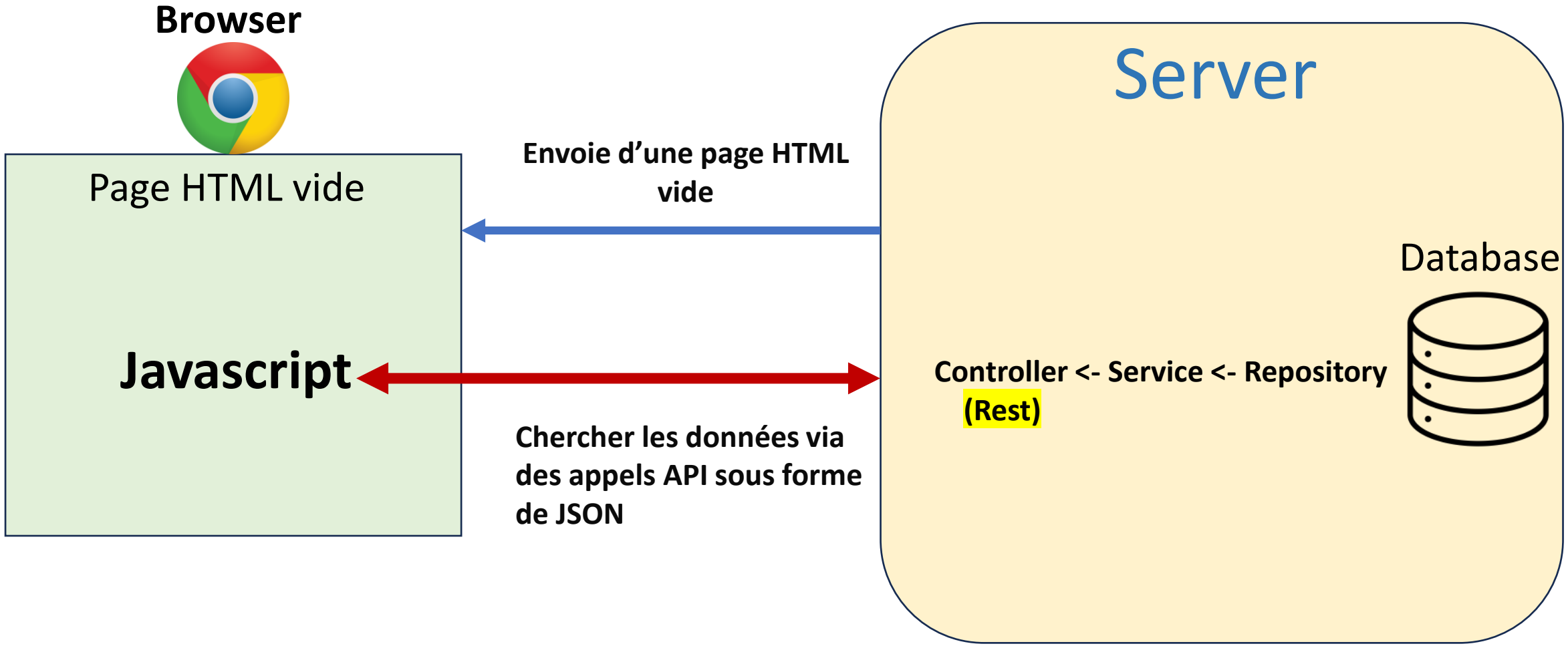
inconvénients :

- ❑ nécessite un rechargement de la page à chaque clic, car le serveur doit renvoyer un nouveau fichier HTML complet.

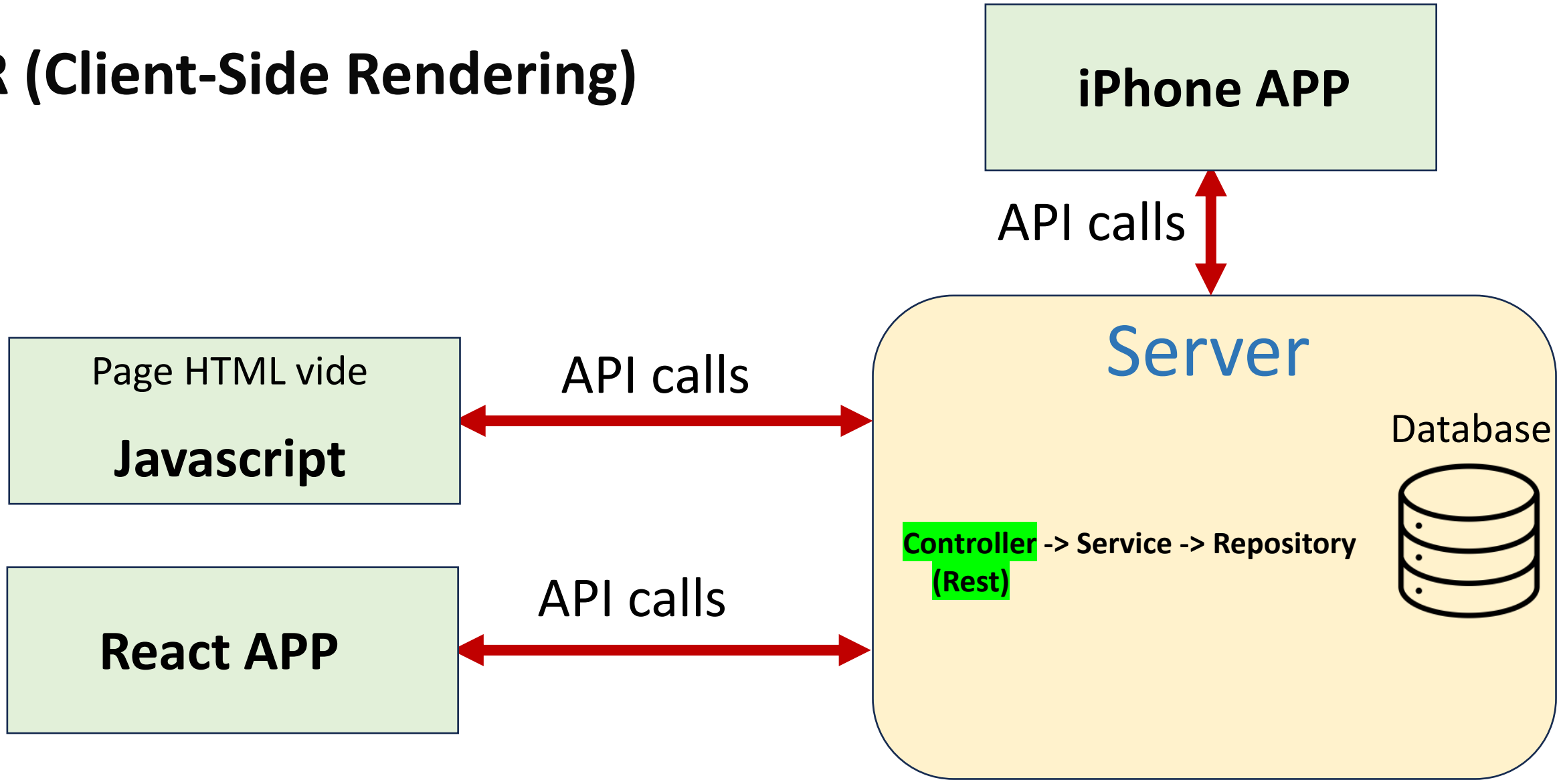
- ❑ Si vous souhaitez créer une application iPhone ultérieurement, vous devrez réécrire toute votre logique, car une application iPhone ne peut pas lire une page HTML.

CSR (Client-Side Rendering)

- Le serveur (Spring Boot) envoie uniquement un fichier HTML presque vide et un fichier JavaScript.



CSR (Client-Side Rendering)



You wrote the logic **once**, and it serves **every device**.

When to use which?

Classique (@Controller) : Idéal pour les blogs simples ou les sites dont le contenu est rarement mis à jour.

Moderne (@RestController + JS) : Indispensable pour toute application souhaitant disposer à terme d'une

- Application mobile
- D'un tableau de bord
- D'une interface en temps réel.

Movie App Demonstration

Page 1 = **Data Fetching Page**

Page with the list of movies being **Dynamily Rendered** via an API.

Page 2 = **Form Submission Page**

Adding a movie via POST request directly to a **View Controller**

- **Hybrid Model App**: Page 1 uses **CSR** to fetch data, while Page 2 uses **SSR** via a standard Spring Boot View Controller.

L’entité Movie

```java
@Entity
@Table(name = "movies")
public class Movie {

@Id
@GeneratedValue(strategy = GenerationType.IDENTITY)
private Long id;
private String title;
private String director;
private Integer rating;

// constructeur vide pour Hibernet
// Getters
// Setters
```

Controller description

Page 1 = Data Fetching Page

Communique avec

MovieAPIController

Page 2 = Form Submission Page

Communique avec

MovieViewController

Src/main/java

└── Com.demo.MovieApp.controllers

- **Views** — MovieViewController
- **API** — MovieAPIController

Methods in Controllers

MovieViewController

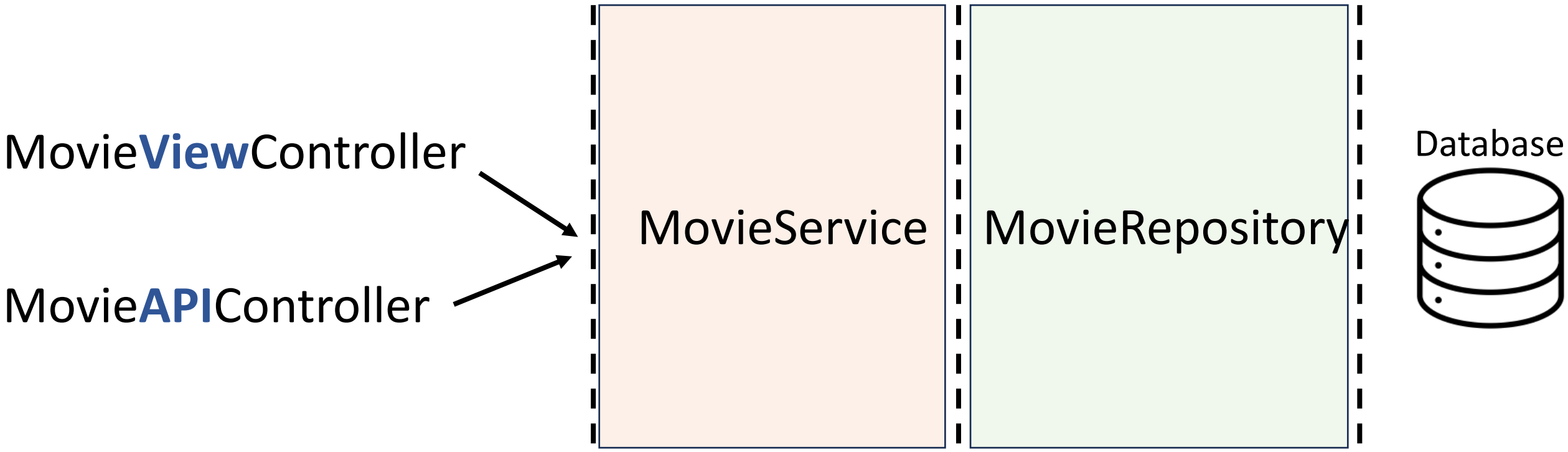
- Afficher une page HTML vide qui sera remplie de données via un appel API. **[public String index()]**
- Afficher une page HTML contenant un formulaire pour ajouter un film. **[public String showForm()]**
- Gérer la logique après que le client a cliqué sur le bouton Créer (requête POST) **[public String submitForm ()]**

MovieAPIController

Interroger la base de données pour charger la liste complète des films.

public List<Movie> getMovies()

Request flow :



- Les deux contrôleurs utilisent les mêmes couches de service et de Repo.

HTML pages :

return

MovieViewController

Index.html

Add-movie.html

Injecter JSON data sans recharger la page via JS

MovieAPIController

Ne devrait pas renvoyer une vue mais uniquement du JSON

Jackson Library

RestController

```
@GetMapping
public List<Movie> getMovies() {
return service.getAllMovies();
}
```

Jackson Library :

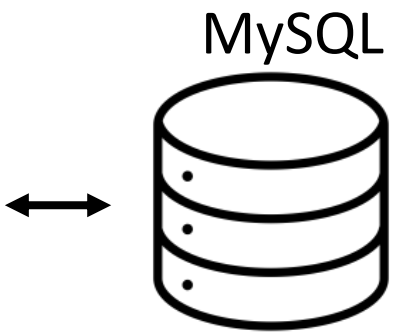
Convert : **Objet** -> **JSON**

Service

```
public List<Movie> getAllMovies() {
return repository.findAll();
}
```

Repository

```
@Repository
public interface MovieRepository extends JpaRepository<Movie, Long>
{
}
```



Difference entre Jackson et res.sjon()

Java Object

In RestController :

Jackson



```
{
  "id": 1,
  "title": "Interstellar",
  "director": "Christopher Nolan",
  "rating": 5
}
```

In javascript :

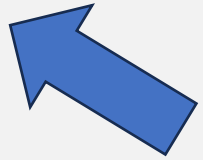
`await res.json()`

Convertissez cette chaîne JSON en un objet JavaScript.

Form submition in Spring boot

La méthode pour afficher le formulaire dans @Controller

```
@GetMapping("/add-movie")
public String showForm(Model model) {
model.addAttribute("movie", new Movie());
return "add-movie";
}
```



- Thymeleaf a besoin de vérifier que les champs existent dans ta classe Java.

- L'objet new Movie() sert de preuve. Si tu fais une faute de frappe, l'application s'arrête tout de suite pour te prévenir.

Structure d'un formulaire Spring boot

```
<form  th:action="@{/add-movie}"
       th:object="${movie}"
       method="post"
       >
```

Controller :

```
model.addAttribute("movie", new Movie());
```

```
<div>

<label> Movie Title </label>

<input type="text"  th:field="*{title}"  placeholder="Enter movie title" />

</div>
```

Vérification

Model :

```
public class Movie {
private String title;
```

```
<button type="submit"></button>
```

```
</form/
```

La méthode qui gère la soumission du formulaire

```
@PostMapping("/add-movie")
public String submitForm(@Valid @ModelAttribute("movie") Movie movie,
BindingResult result) {

if (result.hasErrors()) {
return "add-movie";
}
service.addMovie(movie);
return "redirect:/vault";}
```

- ***@ModelAttribute("movie") Movie movie*** :

Spring Boot prend les données du formulaire, crée un nouvel objet Movie, et appelle automatiquement les setters avec les données saisies.

@Valid

❖Cette annotation demande à Spring de **vérifier les règles de validation**. Si vous avez mis des annotations dans votre classe **`Movie`.**

```
@NotBlank (message = "Title cannot be empty")
private String title;

@Email (message = "Please provide a valid email address")
private String email;

...
```

Exemple

```
@NotBlank (message = "Password is required")
@Size (min = 8, message = "Password must be at least 8 characters long")

private String password;
```

BindingResult result

Si **@Valid** a trouvé des erreurs (ex: Password too short), ces erreurs sont stockées dans cet objet **result**.

```
public String submitForm(@Valid @ModelAttribute("movie") Movie movie,
BindingResult result) {

if (result.hasErrors()) {
return "add-movie";
```

Dans le HTML

```
<p th:if="${#fields.hasErrors('title')}"
th:errors="*{title}" </p>
```

Elle demande au **BindingResult** : **Est-ce qu'il y a un problème avec le champ *title* ? »**

Si la réponse est "Vrai", cette commande va chercher le message d'erreur

API acess

- Pour accéder à la base de données via une API, toute application externe (iPhone, React, Android) doit connaître le **contrat** de votre API.

1. The Base URL :

   localhost/8080 , www.test.com/

2. The Endpoint Path & Method

   To Get Data: GET /api/movies
   To Send Data: POST /api/movies

3. The variable names and data types.
   - **title**: String
   - **director**: String
   - **rating**: Integer

API acess



Front-end Developer

Communiquer les
contrats API





Back-end Developer

```
<dependency>
<groupId>org.springdoc</groupId>
<artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
<version>2.8.5</version>
</dependency>
```

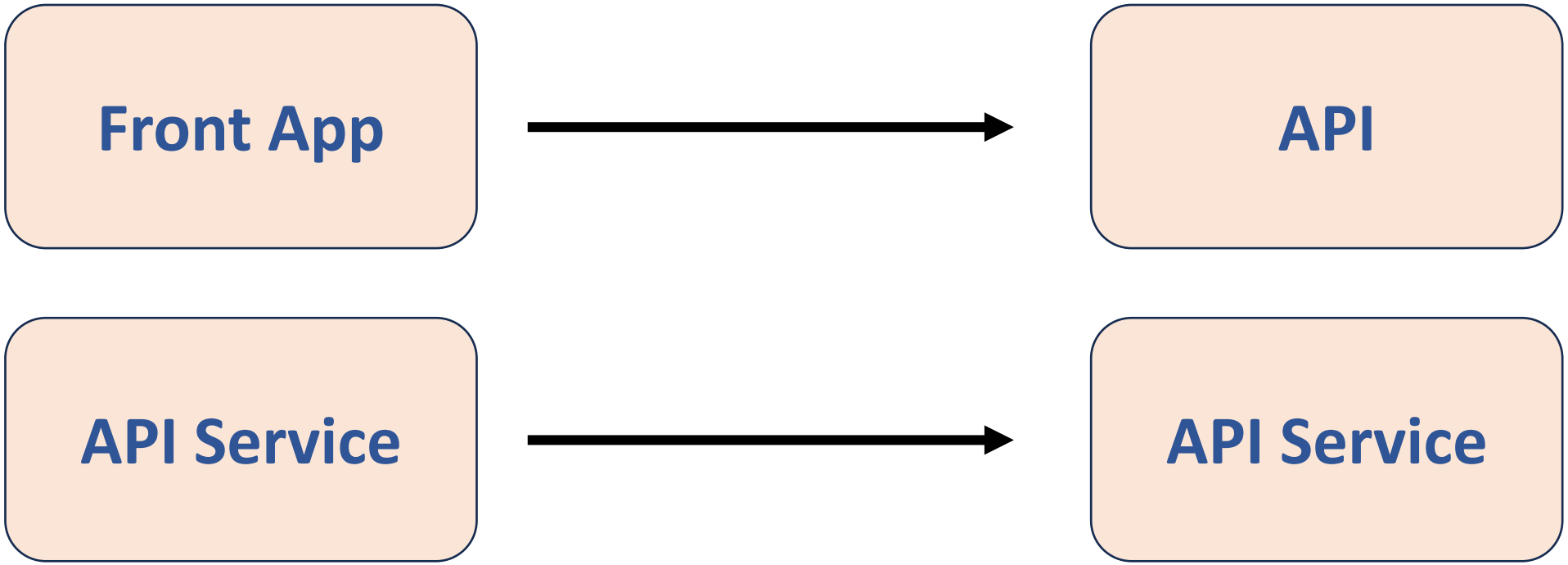
Acess the
contract

http://localhost:8080/swagger-ui/index.html

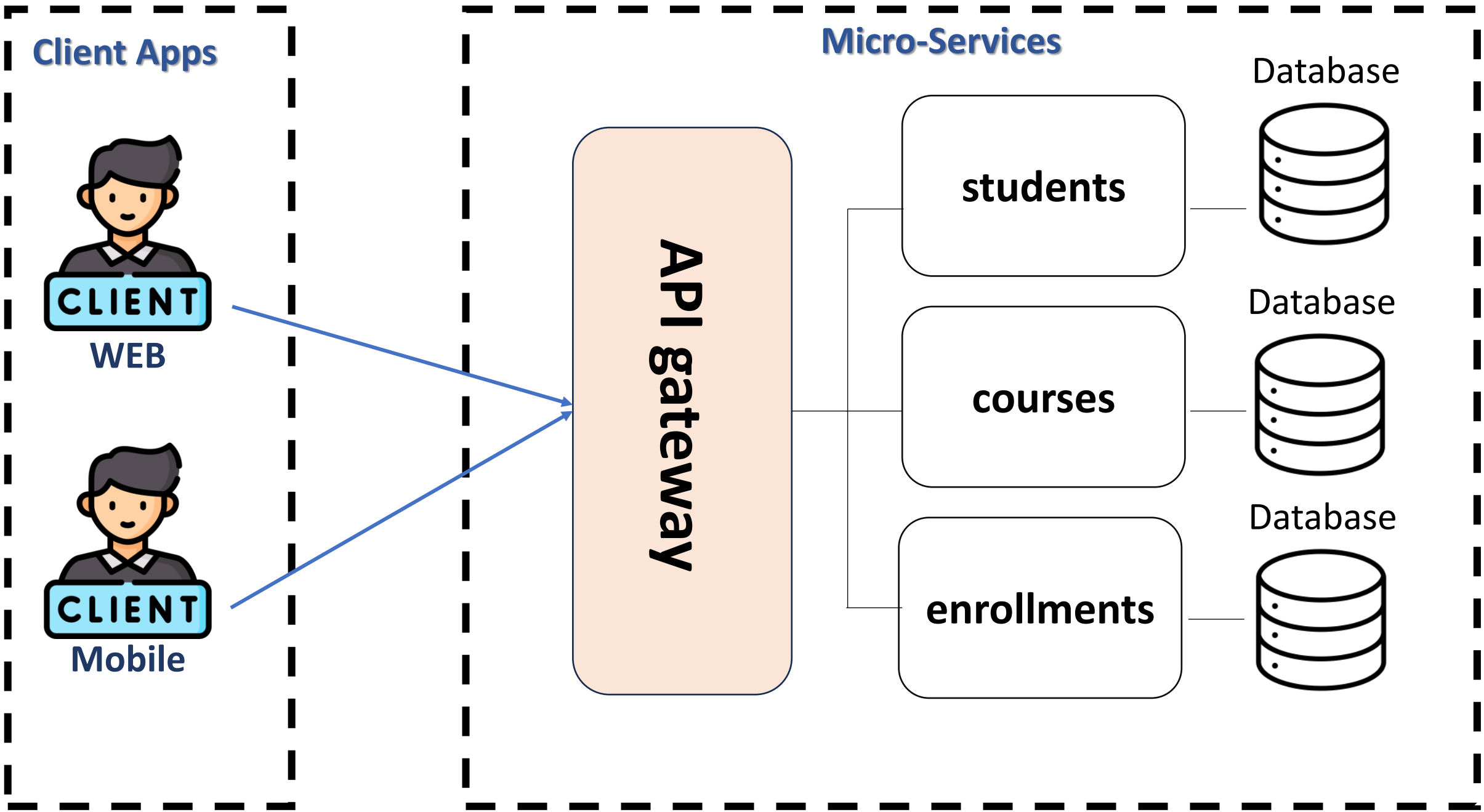
MicroServices and mini-project structure

A microservice system is:

A system where the backend is split into **multiple small independent services**, each responsible for a specific business domain, communicating via APIs.



Microservices-Based Student Enrollment System



System Architecture

The system is composed of four main components:

1. **Student Service:** Manages student identities (Name, Email, and **CNIE**).
2. **Course Service:** Manages the catalog of available subjects.
3. **Enrollment Service:** The "Business Logic" hub that links students to courses.
4. **API Gateway:** The single entry point for the Frontend.

The Enrollment Process – Part 1

- The student enters their **CNIE** (e.g., `CD2387`) and selects a **Course** from a loaded list.
  - ✓ **Logic Constraint :** A course cannot have more than **3 students** enrolled.

- **Backend Validation:** Before saving, the Enrollment Service must:

  - ✓ Verify the student exists (via Student Service).
  - ✓ Verify the course exists (via Course Service).
  - ✓ Check if the current enrollment count for that course is < 3.

The Enrollment Process – Part 2

1) Dashboard

Upon entering their CNIE, the student should see a list of all courses they are currently enrolled in. (The UI must show the **Course Title**)

2) The 24-Hour Cancellation Rule

Students can delete/cancel an enrollment only if it was created **less than 24 hours ago**. The "Delete" button should be disabled or hidden if the 24-hour window has passed.

Technologies In a microservices architecture

Spring WebFlux: A tool that helps the app handle many users at the same time without getting stuck or slow.

Spring Cloud Gateway: The main entry point that receives all requests and sends them to the right service.

Service Discovery (Eureka): A list that keeps track of where each service is running so they can find each other.

WebClient: A tool that allows one service to ask another service for data and combine it into one JSON file.

Student Service Entity

```
@Entity
public class Student {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String cnie;
    private String firstName;
    private String lastName;
    private String email;
}
```

Course Service Entity

```
@Entity
public class Course {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String title;
    private String description;
    private int credits;
}
```

Enrollment Service Entity

```
@Entity
public class Enrollment {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private Long studentId;
    private Long courseId;
    private LocalDateTime enrollmentDate;
}
```

➢ Ce que la base de données d’ **Enrollment** stocke

enrollment_id	student_id	course_id	enrollment_date
45	7	101	2026-04-28 10:30
48	7	202	2026-04-25 14:00

Dans une architecture de microservices, nous évitons la duplication des données. Les données relatives aux cours et aux étudiants doivent se trouver dans leurs bases de données respectives.

The Expected JSON Result in the front-end send by enrolement Service :


```
[
  {
    "enrollmentId": 45,
    "studentCnie": "CD2387",
    "courseName": "Spring Framework",
    "date": "2026-04-28T10:30:00",
    "deletable": true
  },
  {
    "enrollmentId": 48,
    "studentCnie": "CD2387",
    "courseName": "Data Mining",
    "date": "2026-04-25T14:00:00",
    "deletable": false
  }
]
```

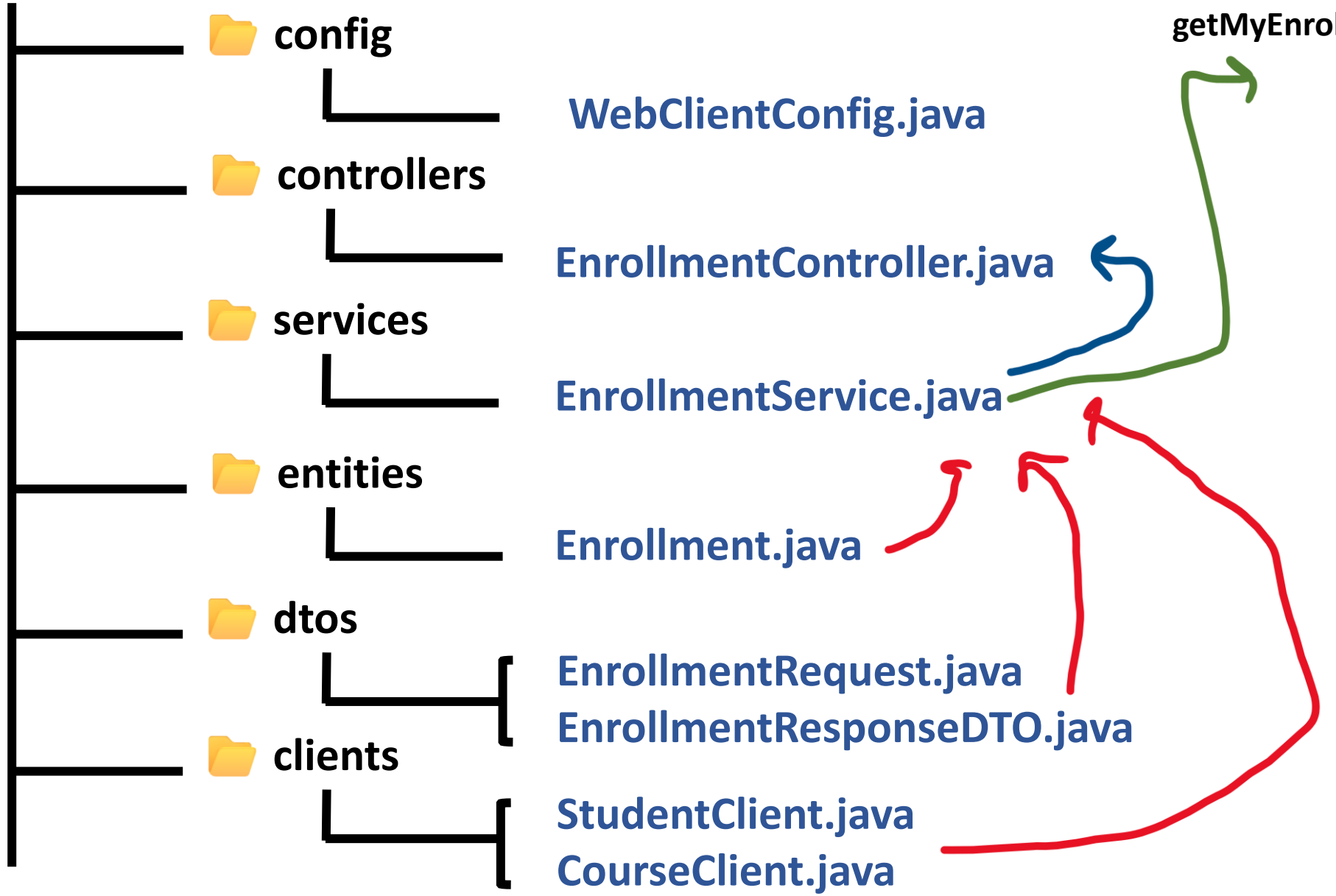
DTO (Data Transfer Object)

- Un DTO est obligatoire car le service «**Enrollment**» combine des données provenant de différents endroits.

```
public class EnrollmentResponseDTO {
    private Long enrollmentId;
    private String studentCnie;
    private String courseName;
    private String date;
    private boolean deletable;

    // Getters and Setters
}
```

src/main/java/com/demo/enrollment



WebClientConfig.java: **The Connection Setup.** It tells the app how to talk to other services using their names instead of fixed addresses.

StudentClient.java: **The Student Finder.** It goes to the Student Service to get the **CNIE** using an ID number.

CourseClient.java: **The Course Finder.** It goes to the Course Service to get the **Course Name** using an ID number.

EnrollmentRequest.java: **The Input Form.** It is a simple object that holds the ID numbers sent from the website when a student clicks "Enroll."

EnrollmentResponseDTO.java: **The Result Package.** It holds all the final info (CNIE + Course Name + Date) in one place to send it back to the website.

Enrollment.java (Entity): **The Database File.** It defines exactly how the enrollment info (ID and Date) is saved in the MySQL table.

EnrollmentRepository.java: **The Database Tool.** It handles the actual saving and searching in the database, like counting how many students are in a class.

EnrollmentService.java: **The Brain.** It holds the rules: it checks if a class has **3 students**, calculates if **24 hours** have passed, and puts all the data together.

EnrollmentController.java: **The Entry Point.** It receives the request from the browser, asks the Service to do the work, and sends the final answer back as JSON.